Your Dreams Our Goal

POORNIMA UNIVERSITY

LECTURE NOTES

UNIT – I

Table of Contents

1.	**Complexity, Memory Allocation, Arrays, and Searching Techniques**
	• Introduction of Unit • Classification of data structures: primitive and non-primitive • Applications of data structures • Time and space complexity of an algorithm • Asymptotic Notations • Memory allocation functions: Malloc(), Calloc(), free() and realloc() • Array Operations • Search Techniques: Sequential search • Iterative and Recursive methods-Binary search • Conclusion of Unit

Introduction to Data Structure

Data structure is a branch of computer science. The study of data structures helps to understand the basic concepts involved in organizing and storing data as well as the relationship among the data sets. This in turn helps to determine the way information is stored, retrieved and modified in a computer's memory. In other words, for many problems the ability to formulate an efficient algorithm depends on being able to organize the data in an appropriate manner. The term *data structure* is used to denote a particular way of organizing data for particular types of operation.

The study of data structure also helps you to understand how data flow is managed to increase efficiency of any process or program. Data structure is the structural representation of logical relationship between data elements. This means that a data structure organizes data items based on the relationship between the data elements.

Need for Data Structure

i. It gives different level of organization data.
ii. It tells how data can be stored and accessed in its elementary level.
iii. Provide operation on group of data, such as adding an item, looking up highest priority item.
iv. Provide a means to manage huge amount of data efficiently.
v. Provide fast searching and sorting of data.

Data structure: A data structure is a specialized format for organizing and storing data. General data structure types include the array, the file, the record, the table, the tree, and so on. Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways.

Algorithms: In this module we will majorly focusing on designing of algorithms/pseudo code to solve different types of problems. In this context, it is essential to learn about nitty-gritty of designing of algorithms first.

Definition*: -** An algorithm is a ***Step By Step process to solve a problem, where each step indicates an intermediate task. Algorithm contains finite number of steps that leads to the solution of the problem.

***Properties /Characteristics of an Algorithm*:-**

i. Algorithm has the following basic properties
ii. Input-Output:- Algorithm takes '0' or more input and produces the required output. This is the basic characteristic of an algorithm.
iii. Finiteness:- An algorithm must terminate in countable number of steps.
iv. Definiteness: Each step of an algorithm must be stated clearly and unambiguously.
v. Effectiveness: Each and every step in an algorithm can be converted in to programming language statement.
vi. Generality: Algorithm is generalized one. It works on all set of inputs and provides the required output. In other words it is not restricted to a single input value.

Categories of Algorithm:

Based on the different types of steps in an Algorithm, it can be divided into three categories, namely

i. Sequence
ii. Selection and
iii. Iteration

Sequence: The steps described in an algorithm are performed successively one by one without skipping any step. The sequence of steps defined in an algorithm should be simple and easy to understand. Each instruction of such an algorithm is executed, because no selection procedure or conditional branching exists in a sequence algorithm.

Example:

// adding two numbers

Step 1: start
Step 2: read a,b
Step 3: Sum=a+b
Step 4: write Sum
Step 5: stop

Selection: The sequence type of algorithms are not sufficient to solve the problems, which involves decision and conditions. In order to solve the problem which involve decision making or option selection, we go for Selection type of algorithm. The general format of Selection type of statement is as shown below:

```
if(condition)
Statement-1;
else
Statement-2;
```

The above syntax specifies that if the condition is true, statement-1 will be executed otherwise statement-2 will be executed. In case the operation is unsuccessful. Then sequence of algorithm should be changed/ corrected in such a way that the system will re-execute until the operation is successful. 2

Iteration: Iteration type algorithms are used in solving the problems which involves repetition of statement. In this type of algorithms, a particular number of statements are repeated 'n' no. of times.

Example1:

Step 1 : start
Step 2 : read n
Step 3 : repeat step 4 until n>0
Step 4 : (a) r=n mod 10
(b) s=s+r
(c) n=n/10
Step 5 : write s
Step 6 : stop

PRACTICE ALGORITHMS:

1.Write an algorithm for roots of a Quadratic Equation?

// Roots of a quadratic Equation
Step 1 : start
Step 2 : read a,b,c
Step 3 : if (a= 0) then step 4 else step 5
Step 4 : Write " Given equation is a linear equation "
Step 5 : d=(b * b) _ (4 *a *c)
Step 6 : if (d>0) then step 7 else step8
Step 7 : Write " Roots are real and Distinct"
Step 8: if(d=0) then step 9 else step 10
Step 9: Write "Roots are real and equal"
Step 10: Write " Roots are Imaginary"
Step 11: stop 3

2. Write an algorithm to find the largest among three different numbers entered by user

Step 1: Start
Step 2: Declare variables a,b and c.
Step 3: Read variables a,b and c.
Step 4: If a>b
If a>c
Display a is the largest number.
Else
Display c is the largest number.
Else
If b>c
Display b is the largest number.
Else
Display c is the greatest number.
Step 5: Stop

3.Write an algorithm to find the factorial of a number entered by user.

Step 1: Start
Step 2: Declare variables n,factorial and i.
Step 3: Initialize variables
factorial←1
i←1
Step 4: Read value of n
Step 5: Repeat the steps until i=n
5.1: factorial←factorial*i
5.2: i←i+1
Step 6: Display factorial
Step 7: Stop

4.Write an algorithm to find the Simple Interest for given Time and Rate of Interest .

Step 1: Start
Step 2: Read P,R,S,T.
Step 3: Calculate S=(PTR)/100
Step 4: Print S
Step 5: Stop

Classification of data structures: primitive and non-primitive

A data structure provides a structured set of variables that are associated with each other in different ways. It forms a basis of programming tool that represents the relationship between data elements and helps programmers to process the data easily.

Data structure can be classified into two categories:

I. Primitive data structure

II. Non-primitive data structure

The following figure 1.1 is showing different types of data structures.

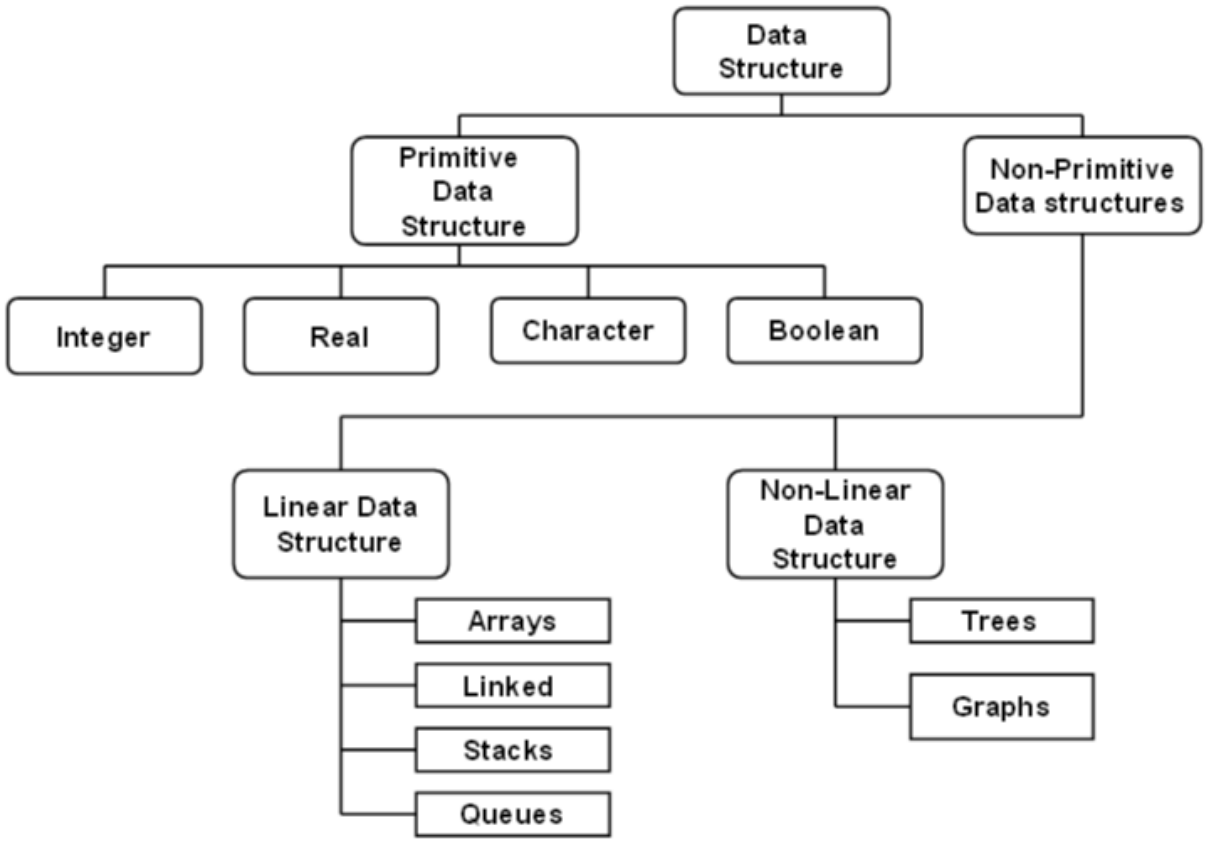


Fig.1.1: Different Types Of Data Structures.

Primitive Data Structure

Primitive data structures consist of the numbers and the characters which are built in programs. These can be manipulated or operated directly by the machine level instructions. Basic data types such as integer, real, character, and Boolean come under primitive data structures. These data types are also known as simple data types because they consist of characters that cannot be divided.

Non-primitive Data Structure

Non-primitive data structures are those that are derived from primitive data structures. These data structures cannot be operated or manipulated directly by the machine level instructions.

They focus on formation of a set of data elements that is either homogeneous (same data type) or heterogeneous (different data type). These are further divided into linear and non-linear data structure based on the structure and arrangement of data.

Linear Data Structure

A data structure that maintains a linear relationship among its elements is called a linear data structure. Here, the data is arranged in a linear fashion. But in the memory, the arrangement may not be sequential. Ex: Arrays, linked lists, stacks, queues.

Non-linear Data Structure

Non-linear data structure is a kind of data structure in which data elements are not arranged in a sequential order. There is a hierarchical relationship between individual data items. Here, the insertion and deletion of data is not possible in a linear fashion.Trees and graphs are examples of non-linear data structures.

I) Array

Array, in general, refers to an orderly arrangement of data elements. Array is a type of data structure that stores data elements in adjacent locations. Array is considered as linear data structure that stores elements of same data types. Hence, it is also called as a linear homogenous data structure.

When we declare an array, we can assign initial values to each of its elements by enclosing the values in braces { }.

int Num [5] = { 26, 7, 67, 50, 66 };

Above declaration will create an array.

The number of values inside braces { } should be equal to the number of elements that we declare for the array inside the square brackets [ ]. In the example of array Paul, we have declared 5 elements and in the list of initial values within braces { } we have specified 5 values, one for each element. After this declaration, array Paul will have five integers, as we have provided 5 initialization values.

Arrays can be classified as one-dimensional array, two-dimensional array or multidimensional array.

- ***One-dimensional Array*:** It has only one row of elements. It is stored in ascending storage location.
- ***Two-dimensional Array:*** It consists of multiple rows and columns of data elements. It is also called as a matrix.
- ***Multidimensional Array:*** Multidimensional arrays can be defined as array of arrays. Multidimensional arrays are not bounded to two indices or two dimensions. They can include as many indices as required.

Limitations:

- Arrays are of fixed size.
- Data elements are stored in contiguous memory locations which may not be always available.
- Insertion and deletion of elements can be problematic because of shifting of elements from their positions. However, these limitations can be solved by using linked lists.

Applications:

- Storing list of data elements belonging to same data type
- Auxiliary storage for other data structures
- Storage of binary tree elements of fixed count

- Storage of matrices

Linked List

A linked list is a data structure in which each data element contains a pointer or link to the next element in the list. Through linked list, insertion and deletion of the data element is possible at all places of a linear list. Also in linked list, it is not necessary to have the data elements stored in consecutive locations. It allocates space for each data item in its own block of memory. Thus, a linked list is considered as a chain of data elements or records called nodes. Each node in the list contains information field and a pointer field. The information field contains the actual data and the pointer field contains address of the subsequent nodes in the list.

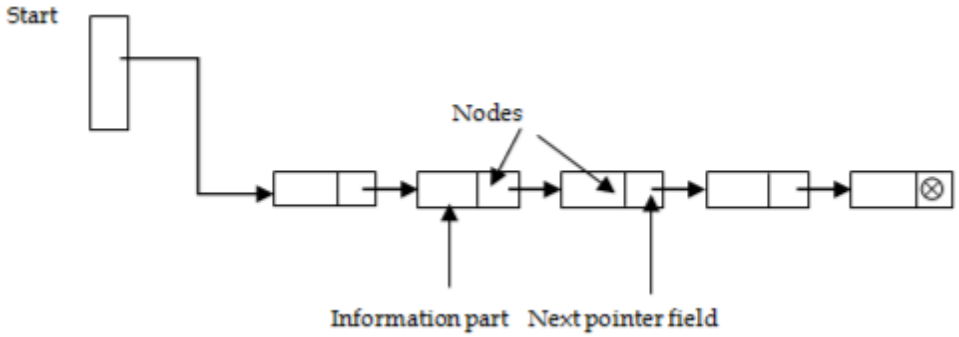


Fig. 1.2: Structure of A Linked list

Figure 1.2 represents a linked list with 4 nodes. Each node has two parts. The left part in the node represents the information part which contains an entire record of data items and the right part represents the pointer to the next node. The pointer of the last node contains a null pointer.

Stacks

A stack is a linear data structure in which insertion and deletion of elements are done at only one end, which is known as the top of the stack. Stack is called a last-in, first-out (LIFO) structure because the last element which is added to the stack is the first element which is deleted from the stack.

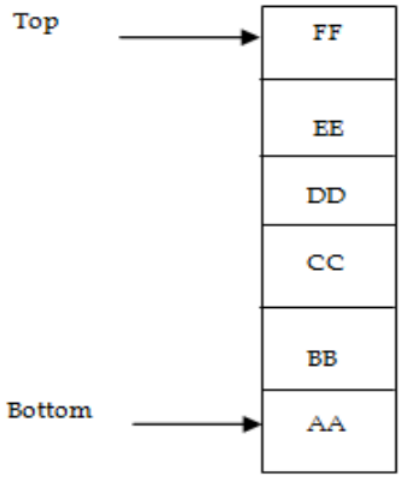


Fig. 1.3: Structure of A Stack

In the computer′s memory, stacks can be implemented using arrays or linked lists. Figure 1.3 is a schematic diagram of a stack. Here, element FF is the top of the stack and element AA is the bottom of the stack. Elements are added to the stack from the top. Since it follows LIFO pattern, EE cannot be deleted before FF is deleted, and similarly DD cannot be deleted before EE is deleted and so on.

Queues

A queue is a first-in, first-out (FIFO) data structure in which the element that is inserted first is the first one to be taken out. The elements in a queue are added at one end called the rear and removed from the other end called the front. Like stacks, queues can be implemented by using either arrays or linked lists. Figure 1.4 shows a queue with 4 elements, where 55 is the front element and 65 is the rear element. Elements can be added from the rear and deleted from the front.

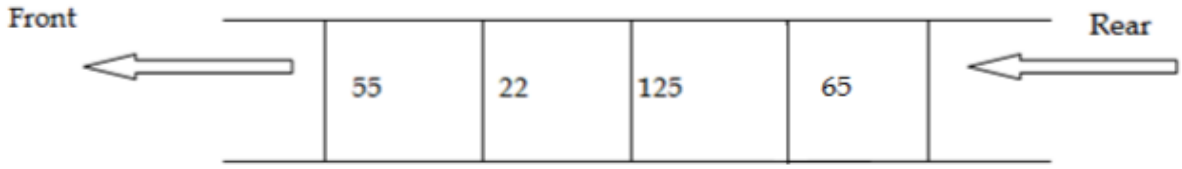


Fig.1.4: Structure of A Queue

Trees

A tree is a non-linear data structure in which data is organized in branches. The data elements in tree are arranged in a sorted order. It imposes a hierarchical structure on the data elements. Figure 1.5 represents a tree which consists of 8 nodes. The root of the tree is the node 60 at the top. Node 29 and 44 are the successors of the node 60. The nodes 6, 4, 12 and 67 are the terminal nodes as they do not have any successors.

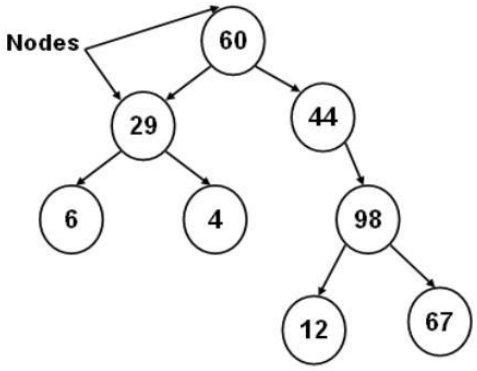


Fig. 1.5: Structure of a Tree

Graphs

A graph is also a non-linear data structure. In a tree data structure, all data elements are stored in definite hierarchical structure. In other words, each node has only one parent node. While in graphs, each data element is called a vertex and is connected to many other vertexes through connections called edges.

Thus, a graph is considered as a mathematical structure, which is composed of a set of vertexes and a set of edges. Figure shows a graph with six nodes A, B, C, D, E, F and seven edges [A, B], [A, C], [A, D], [B, C], [C, F], [D, F] and [D, E].

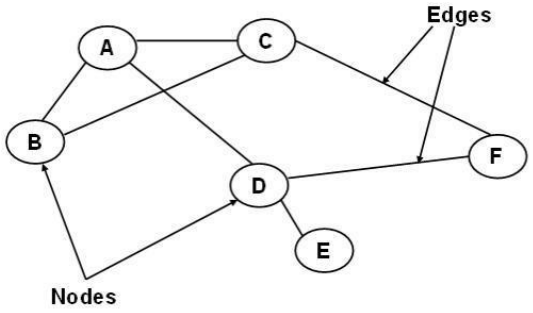


Fig. 1.6: Structure of a Graph

STATIC DATA STRUCTURE VS DYNAMIC DATA STRUCTURE

Data structure is a way of storing and organising data efficiently such that the required operations on them can be performed be efficient with respect to time as well as memory. Simply, Data Structure are used to reduce complexity (mostly the time complexity) of the code. Data structures can be two types:

1. Static Data Structure
2. Dynamic Data Structure

What is a Static Data structure?

In Static data structure the size of the structure is fixed. The content of the data structure can be modified but without changing the memory space allocated to it.

Example

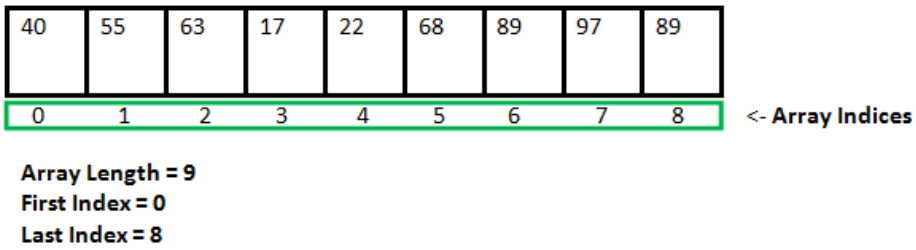


Fig. 1.7: Static Data Structure Array

What is Dynamic Data Structure?

In Dynamic data structure the size of the structure in not fixed and can be modified during the operations performed on it. Dynamic data structures are designed to facilitate change of data structures in the run time.

Fig. 1.7: Static Data Structure Array

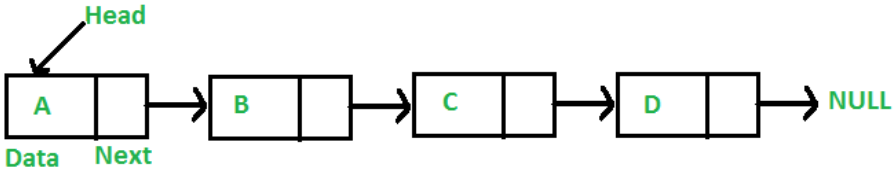


Fig. 1.8: Dynamic Data Structure Linked list

Static Data Structure vs Dynamic Data Structure

Static Data structure has fixed memory size whereas in Dynamic Data Structure, the size can be randomly updated during run time which may be considered efficient with respect to memory complexity of the code. Static Data Structure provides more easier access to elements with respect to dynamic data structure. Unlike static data structures, dynamic data structures are flexible.

OPERATIONS ON DATA STRUCTURES

This section discusses the different operations that can be performed on the various data structures previously mentioned.

Traversing It means to access each data item exactly once so that it can be processed.

For example, to print the names of all the students in a class.

Searching It is used to find the location of one or more data items that satisfy the given constraint. Such a data item may or may not be present in the given collection of data items. For example, to find the names of all the students who secured 100 marks in mathematics.

Inserting It is used to add new data items to the given list of data items. For example, to add the details of a new student who has recently joined the course.

Deleting It means to remove (delete) a particular data item from the given collection of data items. For example, to delete the name of a student who has left the course.

Sorting Data items can be arranged in some order like ascending order or descending order depending on the type of application. For example, arranging the names of students in a class in an alphabetical order, or calculating the top three winners by arranging the participants' scores in descending order and then extracting the top three.

Merging Lists of two sorted data items can be combined to form a single list of sorted data items.

Applications of data structures

The following are Real-Life Applications of Data Structures

a. Arrays: Storing matrices in scientific computing, handling data in databases.

b. Linked Lists: Implementing dynamic memory allocation, managing playlists in media players.

c. Stacks: Undo mechanisms in text editors, function call management in programming.

d. Queues: Job scheduling in operating systems, handling requests in web servers.

e. Trees: File system organization, database indexing, hierarchical data representation.

f. Graphs: Networking (routing), social network analysis, path finding algorithms in maps.

Time and space complexity of an algorithm

Generally, there is always more than one way to solve a problem in computer science with different algorithms. Therefore, it is highly required to use a method to compare the solutions in order to judge which one is more optimal. The method must be:

- Independent of the machine and its configuration, on which the algorithm is running on.
- Shows a direct correlation with the number of inputs.
- Can distinguish two algorithms clearly without ambiguity.

There are two such methods used, **time complexity** and **space complexity** which are discussed below:

Time Complexity: The time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input. Note that the time to run is a function of the length of the input and not the actual execution time of the machine on which the algorithm is running on.

Definition–

The valid algorithm takes a finite amount of time for execution. The time required by the algorithm to solve given problem is called ***time complexity*** of the algorithm. Time complexity is very useful measure in algorithm analysis.

It is the time needed for the completion of an algorithm. To estimate the time complexity, we need to consider the cost of each fundamental instruction and the number of times the instruction is executed.

Example 1: Addition of two scalar variables.

```
Algorithm ADD SCALAR(A, B)
//Description: Perform arithmetic addition of two numbers
//Input: Two scalar variables A and B
//Output: variable C, which holds the addition of A and B
C <- A + B
return C
```

The addition of two scalar numbers requires one addition operation. the time complexity of this algorithm is constant, so $T(n) = O(1)$.

In order to calculate time complexity on an algorithm, it is assumed that a **constant time c** is taken to execute one operation, and then the total operations for an input length on **N** are calculated. Consider an example to understand the process of calculation: Suppose a problem is to find whether a pair **(X, Y)** exists in an array, A of **N** elements whose sum is **Z**. The simplest idea is to consider every pair and check if it satisfies the given condition or not.

The pseudo-code is as follows:

```
int a[n];
for(int i = 0;i < n;i++)
  cin >> a[i]
```

```
for(int i = 0;i < n;i++)
  for(int j = 0;j < n;j++)
    if(i!=j && a[i]+a[j] == z)
      return true
return false
```

Order of growth is how the time of execution depends on the length of the input. Order of growth will help to compute the running time with ease.

Some general time complexities are listed below with the input range for which they are accepted in competitive programming:

f(*n*)	Classification
1	***Constant*: run time is fixed, and does not depend upon n. Most instructions are executed once, or only a few times, regardless of the amount of information being processed**
log n	***Logarithmic*: when *n* increases, so does run time, but much slower. Common in programs which solve large problems by transforming them into smaller problems. Exp : binary Search**
n	***Linear*: run time varies directly with *n*. Typically, a small amount of processing is done on each element. Exp: Linear Search**
n log n	**When *n* doubles, run time slightly more than doubles. Common in programs which break a problem down into smaller sub-problems, solves them independently, then combines solutions. Exp: Merge**
n^2	***Quadratic*: when *n* doubles, runtime increases fourfold. Practical only for small problems; typically the program processes all pairs of input (e.g. in a double nested loop). Exp: Insertion Search**
n^3	***Cubic*: when n doubles, runtime increases eightfold. Exp: Matrix**
2^n	***Exponential*: when n doubles, run time squares. This is often the result of a natural, "brute force" solution. Exp: Brute Force.** **Note: logn, n, nlogn, n^2>> less Input>>Polynomial** **n^3, 2^n>>high input>> non polynomial**

Table: General Time Complexities

Asymptotic Notations: Refer PPT given and discussed in class.

Note: For more detailed study of Time Complexity please refer the ppt provided and discussed to you in the class.

Space Complexity:

Problem-solving using computer requires memory to hold temporary data or final result while the program is in execution. The amount of memory required by the algorithm to solve given problem is called **space complexity** of the algorithm.
The space complexity of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input. Consider an example: Suppose a problem to find the frequency of array elements.

It is the amount of memory needed for the completion of an algorithm.
To estimate the memory requirement we need to focus on two parts:
(1) A fixed part: It is independent of the input size. It includes memory for instructions (code), constants, variables, etc.
(2) A variable part: It is dependent on the input size. It includes memory for recursion stack, referenced variables, etc.

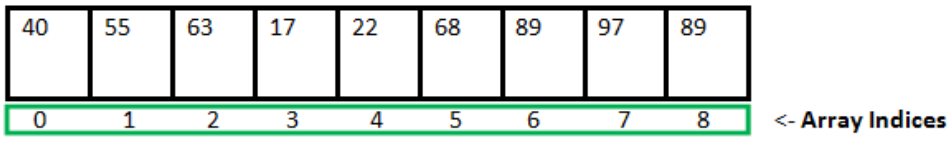
Example : Addition of two scalar variables

```
Algorithm ADD SCALAR(A, B)
//Description: Perform arithmetic addition of two numbers
//Input: Two scalar variables A and B
//Output: variable C, which holds the addition of A and B
C= A+B
return C
```

The addition of two scalar numbers requires one extra memory location to hold the result. Thus the space complexity of this algorithm is constant, hence S(n) = O(1).

Memory allocation functions: malloc(), calloc(), free() and realloc()

Since C is a structured language, it has some fixed rules for programming. One of them includes changing the size of an array. An array is a collection of items stored at contiguous memory locations.



Array Length = 9
First Index = 0
Last Index = 8

As can be seen, the length (size) of the array above is 9. But what if there is a requirement to change this length (size)? For example,

- If there is a situation where only 5 elements are needed to be entered in this array. In this case, the remaining 4 indices are just wasting memory in this array. So there is a requirement to lessen the length (size) of the array from 9 to 5.

- Take another situation. In this, there is an array of 9 elements with all 9 indices filled. But there is a need to enter 3 more elements in this array. In this case, 3 indices more are required. So the length (size) of the array needs to be changed from 9 to 12.

This procedure is referred to as **Dynamic Memory Allocation in C**.

Therefore, C **Dynamic Memory Allocation** can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime.

C provides some functions to achieve these tasks. There are 4 library functions provided by C defined under **<stdlib.h>** header file to facilitate dynamic memory allocation in C programming. They are:

1. malloc()
2. calloc()
3. free()
4. realloc()

Let's look at each of them in greater detail.

C malloc() method

The **"malloc"** or **"memory allocation"** method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form. It doesn't Initialize memory at execution time so that it has initialized each block with the default garbage value initially.

Syntax of malloc() in C

```
ptr = (cast-type*) malloc(byte-size)
For Example:
```

ptr = (int*) malloc(100 * sizeof(int));

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory. If space is insufficient, allocation fails and returns a NULL pointer.

calloc() method

1. **"calloc"** or **"contiguous allocation"** method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but has two different points and these are:
2. It initializes each block with a default value '0'.
3. It has two parameters or arguments as compare to malloc().

Syntax of calloc() in C

```
ptr = (cast-type*)calloc(n, element-size);
here, n is the no. of elements and element-size is the size of each element.
```

For Example:

ptr = (float*) calloc(25, sizeof(float));

This statement allocates contiguous space in memory for 25 elements each with the size of the float. If space is insufficient, allocation fails and returns a NULL pointer.

C free() method

“free” method in C is used to dynamically de-allocate the memory. The memory allocated using functions malloc() and calloc() is not de-allocated on their own. Hence the free() method is used, whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.

```
Syntax of free() in C
free(ptr);
```

C realloc() method

“realloc” or “re-allocation” method in C is used to dynamically change the memory allocation of a previously allocated memory. In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to dynamically re-allocate memory. re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax of realloc() in C

```
ptr = realloc(ptr, newSize);
where ptr is reallocated with new size 'newSize'.
```

Array Operations:

Array, in general, refers to an orderly arrangement of data elements. Array is a type of data structure that stores data elements in adjacent locations. Array is considered as linear data structure that stores elements of same data types. Hence, it is also called as a linear homogenous data structure.
When we declare an array, we can assign initial values to each of its elements by enclosing the values in braces { }.
int Num [5] = { 26, 7, 67, 50, 66 };
Above declaration will create an array.

Arrays can be classified as one-dimensional array, two-dimensional array or multidimensional array.

- ***One-dimensional Array*:** It has only one row of elements. It is stored in ascending storage location.
- ***Two-dimensional Array:*** It consists of multiple rows and columns of data elements. It is also called as a matrix.
- ***Multidimensional Array:*** Multidimensional arrays can be defined as array of arrays. Multidimensional arrays are not bounded to two indices or two dimensions. They can include as many indices as required.

Operations in Arrays

The basic operations in the Arrays are insertion, deletion, searching, display, traverse, and update. These operations are usually performed to either modify the data in the array or to report the status of the array.

Following are the basic operations supported by an array.

- Traverse − print all the array elements one by one.
- Insertion − Adds an element at the given index.
- Deletion − Deletes an element at the given index.
- Search − Searches an element using the given index or by the value.
- Update − Updates an element at the given index.
- Display − Displays the contents of the array.

In C, when an array is initialized with size, then it assigns defaults values to its elements in following order.

Array - Insertion Operation

In the insertion operation, we are adding one or more elements to the array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. This is done using input statements of the programming languages.

Array - Deletion Operation

In this array operation, we delete an element from the particular index of an array. This deletion operation takes place as we assign the value in the consequent index to the current index.

Array - Search Operation

Searching an element in the array using a key; The key element sequentially compares every value in the array to check if the key is present in the array or not.

Array - Traversal Operation

This operation traverses through all the elements of an array. We use loop statements to carry this out.

Array - Update Operation

Update operation refers to updating an existing element from the array at a given index.

Array - Display Operation

This operation displays all the elements in the entire array using a print statement.

Search Techniques: Sequential search

Searching:

Searching is the technique of finding desired data items that has been stored within some data structure. Data structures can include linked lists, arrays, search trees, hash tables, or various other storage methods. The appropriate search algorithm often depends on the data structure being searched.

Search algorithms can be classified based on their mechanism of searching. They are

- Linear searching
- Binary searching

Linear or Sequential searching:

Linear Search is the most natural searching method and It is very simple but very poor in performance at times .In this method, the searching begins with searching every element of the list till the required ecord is found.

The elements in the list may be in any order. i.e. sorted or unsorted. We begin search by comparing the first element of the list with the target element. If it matches, the search ends and position of the element is returned. Otherwise, we will move to next element and compare. In this way, the target element is compared with all the elements until a match occurs. If the match do not occur and there are no more lements to be compared, we conclude that target element is absent in the list by returning position as -1.

For example consider the following list of elements. 55 95 7 5 85 11 25 65 45

Suppose we want to search for element 11(i.e. Target element = 11). We first compare the target element with first element in list i.e. 55. Since both are not matching we move on the next elements in the list and compare. Finally we will find the match after 5 comparisons at position 4 starting from position 0. Linear search can be implemented in two ways

.i) Non recursive

ii) recursive

Algorithm for Linear search

```
Linear_Search (A[ ], N, val , pos )
        Step 1 : Set pos = -1 and k = 0
                Step 2 : Repeat while k < N
                Begin
                Step 3 : if A[ k ] = val
                Set pos = k
                print pos
                Goto step 5
                End while
        Step 4 : print "Value is not present"
        Step 5 : Exit
```

Binary search: It is a fast search algorithm with run-time complexity of O(log n). This search algorithm works on the principle of divide and conquers. Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub-array as well until the size of the sub array reduces to zero. Before applying binary searching, the list of items should be sorted in ascending or descending order. Best case time complexity is O(1) Worst case time complexity is O(log n).

Algorithm:

Binary_Search (A [ ], U_bound, VAL)
Step 1 : set BEG = 0 , END = U_bound , POS = 1
Step 2 : Repeat while (BEG <= END)
Step 3 : set MID = (BEG + END) / 2
Step 4: if A [MID] == VAL then
    POS = MID
    print VAL " is available at ", POS
    GoTo Step 6
End if
if A [MID] > VAL then
    set END = MID – 1
Else
    set BEG = MID + 1
End if
End while
    Step 5 : if POS = -1 then
    print VAL " is not present "
End if
Step 6: EXIT

Recursion:

Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.

Recursion may be a bit difficult to understand. The best way to figure out how it works is to experiment with it.

Example:

The factorial of a positive number n is given by:

```
factorial of n (n!) = 1 * 2 * 3 * 4 *...  * n
```

The factorial of a negative number doesn't exist. And the factorial of 0 is 1. You will learn to find the factorial of a number using recursive and non-recursive function in this example.

Factorial of number (Non.recursive function)

```c
#include <stdio.h>
int main() {
    int n, i;
    unsigned long long fact = 1;
    printf("Enter an integer: ");
    scanf("%d", &n);

    // shows error if the user enters a negative integer
    if (n < 0)
        printf("Error! Factorial of a negative number doesn't exist.");
    else {
        for (i = 1; i <= n; ++i) {
            fact *= i;
        }
        printf("Factorial of %d = %llu", n, fact);
    }

    return 0;
}
```

This program takes a positive integer from the user and computes the factorial using for loop.

Since the factorial of a number may be very large, the type of factorial variable is declared as unsigned long long. If the user enters a negative number, the program displays a custom error message.

Factorial of number (Non.recursive function)

```c
#include<stdio.h>
long int multiplyNumbers(int n);
```

```c
int main() {
    int n;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
    printf("Factorial of %d = %ld", n, multiplyNumbers(n));
    return 0;
}

long int multiplyNumbers(int n) {
    if (n>=1)
        return n*multiplyNumbers(n-1);
    else
        return 1;
}
```